VEGOVA
ELEKTROTEHNIŠKO-RAČUNALNIŠKA
STROKOVNA ŠOLA IN GIMNAZIJA
LJUBLJANA

Splošna matura

Seminarska naloga pri informatiki

Izdelava spletne strani za naročnika

Spletna aplikacija MediForm za enostavno izdelavo poročil o opravljeni zdravstveni negi

Mentor: Aleš Volčini, prof.

Avtor: Lin Čadež, G 4. A

Ljubljana, april 2026

Zahvala

Zahvaljujem se mentorju Alešu Volčiniju za pomoč, usmerjanje in koristne pripombe pri izdelavi seminarske naloge ter pri razvoju aplikacije MediForm.

Posebej se zahvaljujem Heleni Božič Janežič s Srednje zdravstvene šole Ljubljana za naročilo spletne aplikacije MediForm, za vsa njena pojasnila o obstoječem načinu dela, posredovana gradiva in sprotne povratne informacije. Zahvaljujem se tudi dijakom, ki so aplikacijo preizkusili in s svojimi predlogi pomagali izboljšati uporabniško izkušnjo.

Povzetek

V tej seminarski nalogi predstavljam razvoj spletne aplikacije MediForm, ki sem jo izdelal za Srednjo zdravstveno šolo Ljubljana. Aplikacija je namenjena dijakom zdravstvene nege za sprotno in strukturirano izpolnjevanje poročil o obravnavah na področju zdravstvene nege. Za digitalno rešitev sem se odločil zato, ker so bila poročila, ki so jih dijaki prej izpolnjevali v Wordu, zamudna, neenotna in pogosto izpolnjena šele tik pred rokom oddaje, kar je povzročalo, da so si dijaki podatke enostavno izmislili. V nalogi opisujem analizo potreb naročnika, potek dogovarjanja, izbiro tehnologij ter tehnično izvedbo ključnih funkcionalnosti, pri čemer posebno pozornost namenjam varovanju osebnih podatkov in skladnosti z uredbo GDPR.

Ključne besede: spletna aplikacija, MediForm, zdravstvena nega, praktični pouk, React, TypeScript, Tailwind CSS, lokalno shranjevanje, GDPR v zdravstvu

Abstract

This thesis presents the development of MediForm, a web application created for the Secondary School of Nursing in Ljubljana. The application enables nursing students to complete structured reports about patient care activities directly on their mobile phones or computers. The project was initiated in response to issues with the existing Word-based system, where students frequently filled in reports retroactively and fabricated data. The thesis describes the client's needs analysis, the negotiation process, the choice of technologies, and the technical implementation of key functionalities, with particular attention to personal data protection and GDPR compliance.

Keywords: web application, MediForm, healthcare, nursing education, React, TypeScript, Tailwind CSS, local storage, GDPR

Seznam kratic in okrajšav

API Application Programming Interface (vmesnik za programiranje aplikacij)

CSS Cascading Style Sheets (kaskadne stilske predloge)

GDPR General Data Protection Regulation (Splošna uredba o varstvu podatkov)

HTML HyperText Markup Language (jezik za označevanje nadbesedila)

JSON JavaScript Object Notation (format za izmenjavo podatkov)

JS JavaScript

PDF Portable Document Format

React Odprtokodna knjižnica za gradnjo uporabniških vmesnikov

TS TypeScript

TTF TrueType Font (format pisave)

UI User Interface (uporabniški vmesnik)

URL Uniform Resource Locator (enotni lokator vira)

Kazalo vsebine

1 Uvod **7**

2 Potrebe naročnika **8**

2.1 Obstoječi način dela in težave 8

2.2 Proces dogovarjanja 9

3 Aplikacija MediForm **10**

3.1 Kaj aplikacija dela 10

3.2 Izbira tehnologij 11

3.2.1 Tehnološki sklad 11

3.2.2 Zakaj frontend-only rešitev 12

3.2.3 Primer importov in namen knjižnic 12

3.3 Arhitektura in struktura projekta 13

3.4 Tehnična izvedba ključnih funkcionalnosti 15

3.4.1 Struktura obrazca in izbira podatkov 15

3.4.2 Shranjevanje stanja obrazca in localStorage 15

3.4.3 Prikaz napredka obrazca 17

3.4.4 Generiranje PDF in varovanje osebnih podatkov pri izvozu 19

3.4.5 Odzivni prikaz na telefonu 22

3.5 Hranjenje podatkov, GDPR in pravne zadeve 22

3.5.1 Splošno o shranjevanju 22

3.5.2 Skladnost z uredbo GDPR 23

3.5.3 Delovanje localStorage in upravljanje z veljavnostjo dokumentov 24

3.6 Testiranje aplikacije 24

3.7 Objava spletne strani 24

4 Zaključek **25**

Slike

1 Primer originalnega obrazca, ki so ga dijaki izpolnjevali prej (izrez iz Wordove predloge) 8
2 Vsebina originalnega obrazca — primer izpolnjenih podatkov 9
3 Domača stran aplikacije MediForm 11
4 Prijavna stran aplikacije MediForm 13
5 Primer spustnega seznama za izbiro podatkov o pacientu 16
6 Primer večkratne izbire aktivnosti zdravstvene nege 17
7 Prikaz dokončanega dela obrazca, kjer sistem z zeleno kljukico označi zaključen odsek 19
8 Možnosti izvoza poročila 19
9 Primer generiranega PDF poročila 21

1 Uvod

Za to seminarsko nalogo sem se odločil, ker je profesorica Helena Božič s Srednje zdravstvene šole Ljubljana želela, da bi obstajal digitalni način, ki bi poenostavil ustvarjanje poročil njenih dijakov, ki odhajajo na prakso v bolnišnice. Skupaj smo ugotovili, da se pri izpolnjevanju teh poročil pogosto pojavljajo težave, saj jih dijaki izpolnjujejo prepozno, površno in neenotno, kar otežuje preglednost, spremljanje napredka ter delo mentorjev. Zaradi tega sem želel raziskati, kako lahko digitalne rešitve pomagajo izboljšati ta proces.

Danes se šole vedno bolj usmerjajo v digitalizacijo, saj ta omogoča hitrejše delo, boljšo organizacijo podatkov in manj napak. Digitalni obrazci lahko nadomestijo ročno izpolnjevanje ter omogočijo bolj strukturiran in pregleden vnos podatkov. Kljub temu pa ostaja odprto vprašanje, kako zasnovati tak sistem, da bo enostaven za uporabo in hkrati dovolj učinkovit za potrebe zdravstvenega izobraževanja.

Namen naloge je bil razviti spletno aplikacijo, ki omogoča enostavno in sprotno izpolnjevanje poročil o zdravstveni negi. Pri tem sem se osredotočil na preglednost, enotnost zapisov in varno shranjevanje podatkov. Raziskal sem tudi, katere tehnologije so najbolj primerne za takšno rešitev in zakaj.

Cilji seminarske naloge so bili:

- analizirati obstoječe stanje in potrebe uporabnikov,
- razviti delujočo spletno aplikacijo za izdelavo poročil,
- razložiti uporabljene tehnologije in način delovanja,
- zagotoviti varno ravnanje z občutljivimi podatki (GDPR),
- prikazati potek razvoja in sodelovanja s stranko.

Pri delu sem uporabil metodo analize problema, načrtovanja rešitve ter praktične implementacije aplikacije. Poleg tega sem testiral delovanje sistema in preverjal, ali rešitev dejansko izboljša proces izpolnjevanja poročil.

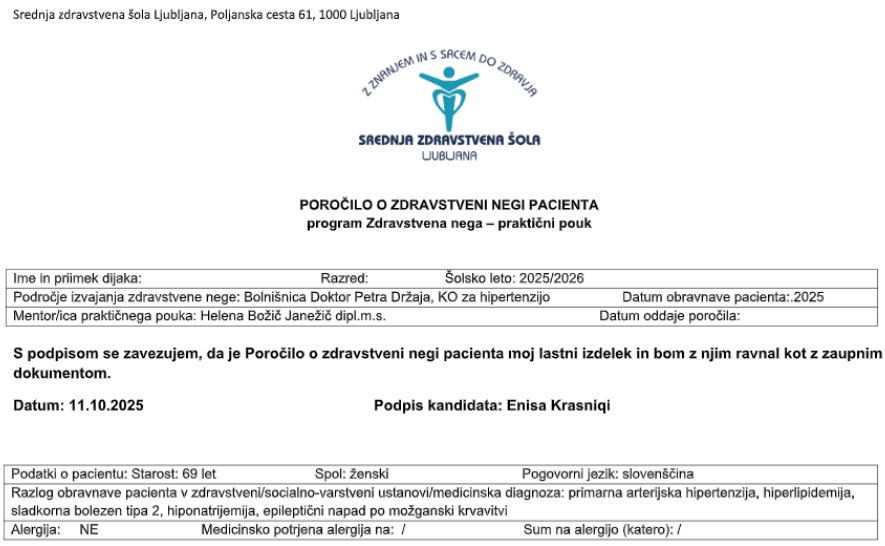
2 Potrebe naročnika

Pred začetkom razvoja aplikacije sem moral natančno razumeti način dela naročnice, njene težave pri obstoječem postopku ter pričakovanja glede nove digitalne rešitve.

2.1 Obstoječi način dela in težave

Ko sem prvič stopil v stik s Heleno Božič Janežič s Srednje zdravstvene šole Ljubljana, mi je predstavila način izpolnjevanja poročil. Dijaki so uporabljali Word predlogo, ki jo je pripravila mentorica, vendar je tak način povzročal težave.

Ker so dijaki poročila pogosto pisali tik pred oddajo, so si podatke izmislili ali jih zapisali po spominu. Zato poročila niso odražala dejanskega stanja pri pacientu, ampak so bila le formalnost.



Srednja zdravstvena šola Ljubljana, Poljanska cesta 61, 1000 Ljubljana

Z ZNANJEM IN S SRCEM DO ZDRAVJA

SREDNJA ZDRAVSTVENA ŠOLA LJUBLJANA

POROČILO O ZDRAVSTVENI NEGI PACIENTA
program Zdravstvena nega – praktični pouk

Ime in priimek dijaka:	Razred:	Šolsko leto: 2025/2026
Področje izvajanja zdravstvene nege: Bolnišnica Doktor Petra Držaja, KO za hipertenzijo		Datum obravnave pacienta:.2025
Mentor/ica praktičnega pouka: Helena Božič Janežič dipl.m.s.		Datum oddaje poročila:

S podpisom se zavezujem, da je Poročilo o zdravstveni negi pacienta moj lastni izdelek in bom z njim ravnal kot z zaupnim dokumentom.

Datum: 11.10.2025 **Podpis kandidata: Enisa Krasniqi**

Podatki o pacientu: Starost: 69 let	Spol: ženski	Pogovorni jezik: slovenščina
Razlog obravnave pacienta v zdravstveni/socialno-varstveni ustanovi/medicinska diagnoza: primarna arterijska hipertenzija, hiperlipidemija, sladkorna bolezen tipa 2, hiponatrijemija, epileptični napad po možganski krvavitvi		
Alergija: NE	Medicinsko potrjena alergija na: /	Sum na alergijo (katero): /

Slika 1: Primer originalnega obrazca, ki so ga dijaki izpolnjevali prej (izrez iz Wordove predloge)

Ključne težave obstoječega sistema so bile:

- zapisi so bili pogosto površni in izmišljeni tik pred oddajo,
- nekateri deli poročil so se ponavljali in bili neenotni,

Srednja zdravstvena šola Ljubljana, Poljanska cesta 61, 1000 Ljubljana

DIHANJE IN KRVNI OBTOK - ocena stopnje samooskrbe pacienta			
Pulz	**Pulz (frekvenca)**: 72 utripov/min normokardija **Ritem:** enakomeren **Polnjenost**: dobro polnjen/dobro tipljiv **Vrsta meritve**: palpacija		
Krvni tlak	**Krvni tlak (izmerjena vrednost)**: 164/67 mmHg hipertenzija **Vrsta meritve:** elektronski merilec		
Dihanje	**Dihanje (frekvenca)**: 16 vdihov/min evpneja **Ritem dihanja**: enakomeren **Globina dihanja**: primerno globoko **Način dihanja:** diha skozi nos **Motnje dihanja: /** **Kašelj**: / **Sputum**: / **Vrsta meritve**: inspekcija (opazovanje)		
SpO2	**Izmerjena vrednost SpO_2 (brez aplikacije kisika)**: 98% pričakovana vrednost **Izmerjena vrednost s SpO_2 (pri aplikaciji kisika): /**		
Aplikacija kisika	**Predpisana koncentracija O_2**: / **Izbrani pripomoček za aplikacijo kisika**: /		
Razvade	Nekadilec		
Aktivnosti zdravstvene nege pri življenjski aktivnosti dihanje in krvni obtok			
Aktivnosti zdravstvene nege	Podatki o izvedeni aktivnosti zdravstvene nege	Datum izvedbe: 7.10.2025	
Identifikacija pacienta	**Identifikacija pravega pacienta (vprašala sem po imenu, priimku, datumu rojstva, preverila sem zapestnico in dokumentacijo).**		
Ocena stopnje samooskrbe v zvezi z dihanjem in krvnim obtokom	**Pridobila z opazovanjem, meritvami, pogovorom s pacinetko in sobno medicinsko sestro. Z meritvami in opazovanjem sem ugotovila, da ima pacientka normokardijo, evpnejo in sistolično hipertenzijo.**		
Merjenje dihanja	**Z opazovanjem, 16 vdihov/min (evpneja-normalno).**		
Merjenje krvnega tlaka	**Z elektronskim merilcem, 164/67 mmHg (hipertenzija-povišano), upoštevala sem vsa načela za pravilno merjenje RR. Pri odstopanju iz normalne vrednosti sem ponovila meritev in obvestila sobno medicinsko sestro o odstopanju.**		

Slika 2: Vsebina originalnega obrazca — primer izpolnjenih podatkov

- sprotnega preverjanja skoraj ni bilo,
- pisanje poročila brez vnaprej ponujenih možnosti je bilo za dijake zamudno in nepraktično na terenu (večinoma so jih pisali kasneje na računalniku).

2.2 Proces dogovarjanja

Proces dogovarjanja sem razdelil na več faz. Najprej sem prejel obrazce in skupaj z naročnico pregledal, kateri deli so potrebni in katere bi bilo mogoče poenostaviti. Na tej osnovi sem zasnoval prvo različico aplikacije.

Po začetku uporabe sem prejel prve povratne informacije. Pojavile so se težave pri prijavi, zato sem izboljšal shranjevanje podatkov in stabilnost delovanja. Naročnica je poudarila, da mora biti aplikacija prilagojena telefonu, omogočati sprotno izpolnjevanje in poenostaviti obrazce.

Ključne zahteve so bile:

- ponavljajoče podatke rešiti z izbirnimi seznami,
- omogočiti hitro in enostavno izpolnjevanje,

- zagotoviti delovanje na različnih napravah (predvsem telefonih),
- omogočiti sprotno shranjevanje podatkov,
- ustrezno zaščititi zasebnost pacientov.

Aplikacijo so najprej testirale izbrane dijakinje, nato pa se je uporaba razširila na celoten oddelek. Vsebino sem sproti prilagajal glede na njihove povratne informacije.

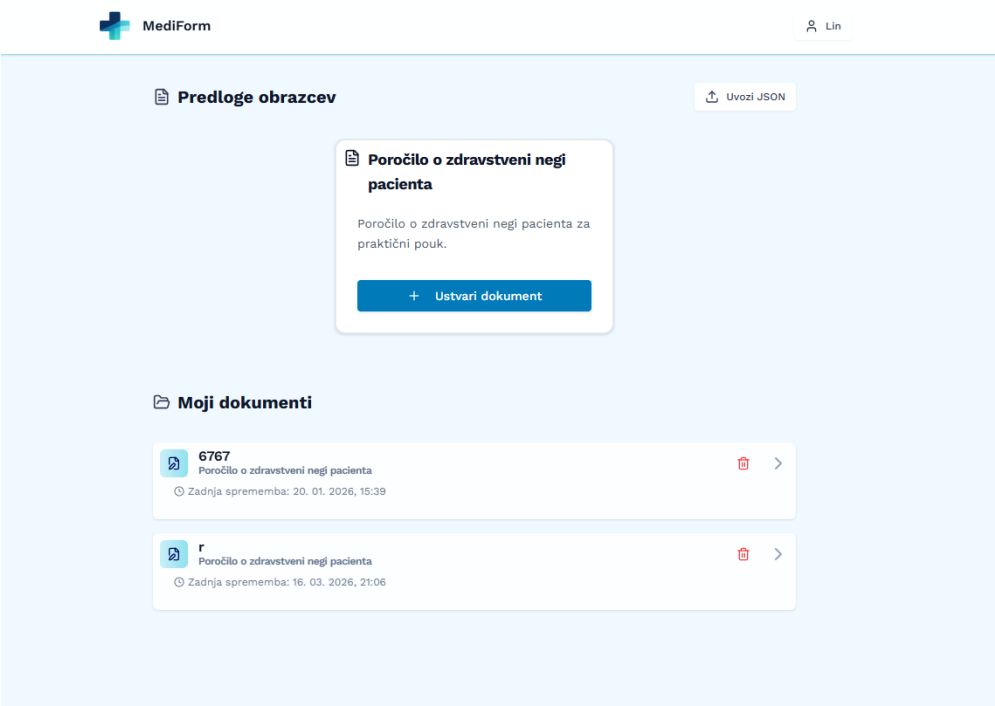
Na predlog mentorja Aleša Volčinija sem posebno pozornost namenil tudi varstvu osebnih podatkov, saj aplikacija obdeluje občutljive informacije.

3 Aplikacija MediForm

Na podlagi ugotovljenih potreb sem zasnoval in izdelal spletno aplikacijo MediForm, s katero rešujem glavne težave obstoječega načina dela. V nadaljevanju bom podrobneje predstavil uporabljene tehnologije, strukturo projekta ter tehnično izvedbo ključnih funkcionalnosti.

3.1 Kaj aplikacija dela

MediForm je spletna aplikacija, ki dijakom zdravstvene nege omogoča strukturirano izpolnjevanje poročil o praktičnem pouku neposredno na telefonu ali računalniku. Namesto ročnega pisanja dijak izbira med vnaprej pripravljenimi možnostmi, vnaša ključne meritve in ob koncu izvozi poročilo v obliki PDF datoteke. Če pa želi ohraniti strukturo obrazca, lahko poročilo shrani tudi v obliki JSON datoteke, ki jo lahko kasneje ponovno naloži in nadaljuje z izpolnjevanjem na drugi napravi.



Slika 3: Domača stran aplikacije MediForm

3.2 Izbira tehnologij

3.2.1 Tehnološki sklad

Pri razvoju sem se odločil za naslednji tehnološki sklad: React in TypeScript za uporabniški vmesnik, Tailwind CSS za oblikovanje, Vite za hiter razvoj in lokalno testiranje ter localStorage in piškotke za shranjevanje stanja v brskalniku.

React je knjižnica za gradnjo vmesnikov iz manjših, samostojnih komponent. Prednost je v ponovni uporabnosti in preglednosti kode. Slabost je, da zahteva določeno predznanje in je za enostavne projekte lahko preobsežen. V tem projektu je bila primerna izbira, ker ima obrazec veliko ponavljajočih se gradnikov.

TypeScript je nadgradnja JavaScripta z dodanimi tipi. Prednost je, da se napake odkrijejo že pri pisanju kode in ne šele pri izvajanju. Slabost je nekoliko več pisanja. Pri projektu s kompleksnimi obrazci je prispeval k stabilnosti.

Tailwind CSS je ogrodje za oblikovanje, ki temelji na razredih. Prednost je hitrost oblikovanja brez pisanja ločenih CSS datotek. Slabost je, da so HTML elementi videti polni razredov, vendar enotstno oblikovanja, ki ji omogoča, to pretehta.

Vite je orodje za lokalni razvoj, ki omogoča takojšnje osveževanje strani ob spremembi kode. To je bistveno pospešilo razvoj.

3.2.2 Zakaj frontend-only rešitev

V zgodnji fazi projekta sem preverjal tudi možnosti shranjevanja podatkov in integracijo sistema prijave na storitvi Firebase, vendar sem to opustil. Mentor je predlagal, da je za paciente varneje, če se podatki shranjujejo izključno lokalno. Takšen način zasnove aplikacije omogoča tudi večjo stabilnost, saj je edini dejavnik, ki lahko vpliva na nedosegljivost aplikacije, morebitna nedosegljivost ponudnika gostovanja (v mojem primeru Vercela). Poleg tega ima Srednja zdravstvena šola v prihodnosti zelo enostavno možnost, da spletno aplikacijo gosti na lastnih strežnikih, brez zapletenega postopka vzpostavitve podatkovne baze ali drugih dodatnih sistemov.

3.2.3 Primer importov in namen knjižnic

Izsek kode 1 prikazuje uvozne stavke, ki pokažejo, katere knjižnice so v uporabi.

```
import { useState, useEffect } from "react";
import {
  BrowserRouter as Router,
  Route,
  Routes,
} from "react-router-dom";
import CookieConsent from "./components/CookieConsent";
import Checklist from "./checklist/checklist";
import { PDFDocument, rgb } from "pdf-lib";
import fontkit from "@pdf-lib/fontkit";
```

Izsek kode 1: Uvozi (importi) v glavni datoteki App.tsx

Knjižnica `react` omogoča gradnjo vmesnika s komponentami in upravljanje stanja (`useState`, `useEffect`). Knjižnica `react-router-dom` skrbi za prehode med stranmi. Komponenta `CookieConsent` upravlja soglasje za piškotke, `Checklist` pa je osrednji del za izpolnjevanje poročila. Knjižnici `pdf-lib` in `@pdf-lib/fontkit` sta namenjeni generiranju PDF poročil.

3.3 Arhitektura in struktura projekta

Aplikacijo sem zasnoval kot frontend-only rešitev. Vse bistvene funkcionalnosti potekajo v brskalniku, podatki pa se shranjujejo lokalno.

Ob prvem obisku spletne strani se uporabniku najprej prikaže prijavna stran, kjer vnese osnovne osebne podatke, kot so e-pošta, ime, priimek, razred, šola ter področje oziroma ustanova. Po uspešnem vnosu se odpre nadzorna plošča, kjer so prikazani vsi obrazci, shranjeni v `localStorage`. Na dnu prijavne strani sta na voljo tudi podstrani `/pogoji-uporabe` in `/zasebnost`, ki sem ju dodal zaradi skladnosti z zakonodajo.

Slika 4: Prijavna stran aplikacije MediForm

Osnovna JSON struktura obrazca je shranjena v datoteki `public/cached-forms/zdravstvena-nega.json`, iz katere aplikacija ob zagonu prebere definicije polj, skupin in vnaprej določenih možnosti ter na tej osnovi dinamično generira uporabniški obrazec. Datoteka `index.json` v isti mapi določa seznam razpoložljivih obrazcev.

Mapa `public/fonts` vsebuje pisave, ki se uporabljajo pri izvozu PDF dokumentov, da so slovenski znaki pravilno prikazani tudi v generiranem poročilu. Skripta `scripts` je namenjena pomožnim opravilom, kot so pretvorba podatkov, priprava predpomnjenih obrazcev, popravljanje strukture JSON datotek.

Glavni del aplikacije se nahaja v mapi `src`. Datoteka `App.tsx` povezuje glavno logiko aplikacije, `main.tsx` pa poskrbi za zagon aplikacije v brskalniku. V mapi `checklist` so komponente in logika za izpolnjevanje obrazca, torej za prikaz vprašanj, podatki za prikaz polj z večkratno in enkratno izbiro ter generiranje PDF poročila. Datoteka `pdfGenerator.tsx` je namenjena pripravi končnega dokumenta.

Spodaj je prikazana struktura datotek projekta:

```
MediForm/
|-- public/
|   |-- cached-forms/
|   |   |-- index.json
|   |   `-- zdravstvena-nega.json
|   `-- fonts/
|       |-- Roboto-Bold.ttf
|       `-- Roboto-Regular.ttf
|-- scripts/          (26 pomoznih skript za testiranje delovanja)
`-- src/
    |-- App.tsx
    |-- main.tsx
    |-- checklist/
    |   |-- checklist.tsx
    |   |-- pdfGenerator.tsx
    |   |-- MultiSelectInput.tsx
    |   `-- SingleSelectComponent.tsx
    |-- components/
    |   |-- CookieConsent.tsx
    |   |-- UserInfoForm.tsx
    |   |-- UserLogin.tsx
    |   `-- ui/
    |-- profil/
    |   |-- profil.tsx
    |   |-- Privacy.tsx
    |   `-- Terms.tsx
    `-- selector/
        `-- selector.tsx
```

Mapa `components` vsebuje splošne uporabniške komponente, kot so prijava uporabnika, vnos osnovnih podatkov, soglasje za piškotke. Podmapa `components/ui` vsebuje osnovne gradnike uporabniškega vmesnika, na primer gumbe, polja, dialoge in druge elemente, ki se večkrat uporabijo.

V mapi `profil` so podstrani, ki pokrivajo pravni in informativni del aplikacije, predvsem `Privacy.tsx` in `Terms.tsx`. Mapa `selector` vsebuje zaslon oziroma komponento za izbor obrazca, kjer uporabnik izbere, katero poročilo želi izpolnjevati.

3.4 Tehnična izvedba ključnih funkcionalnosti

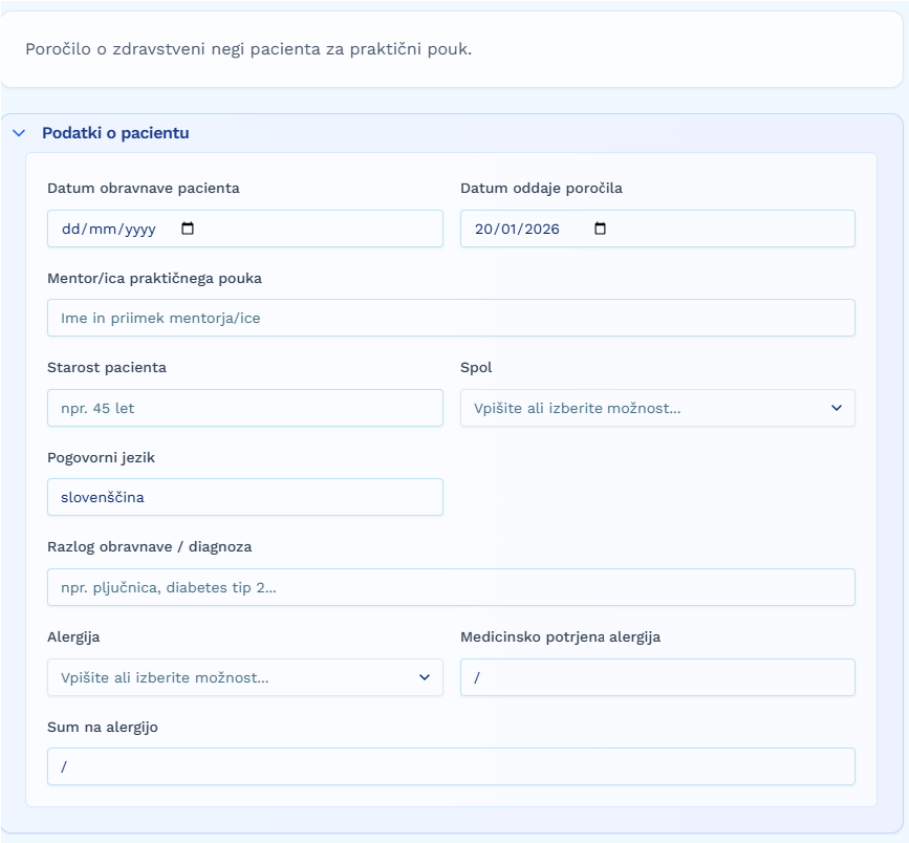
3.4.1 Struktura obrazca in izbira podatkov

Moj kontrolni seznam podpira več načinov vnosnih polj: prosto besedilno polje (`string`), številski vnos (`number`), potrditveno polje (`boolean`, izbira da/ne), datum (`date`, ki ima vgrajen tudi izbirnik datuma), tabelo (za zapise, kot so na primer aplikacije zdravil) ter izbiro z enkratnim ali večkratnim izborom (`single select` in `multiple select`).

Pri enkratnem izboru lahko uporabnik izbere eno izmed vnaprej pripravljenih možnosti ali vnese lastno vrednost, medtem ko lahko pri večkratnem izboru izbere več možnosti in po potrebi doda tudi svoje.

3.4.2 Shranjevanje stanja obrazca in localStorage

```
// Shranjevanje stanja: expiration = 60 dni (približno 2 meseca)
const saveFormState = (formId, formState) => {
  const expirationDate =
    Date.now() + 60 * 24 * 60 * 60 * 1000; // 60 dni v ms
  const dataToStore = {
    state: formState,
    expires: expirationDate,
  };
  localStorage.setItem(
    `mediform-${formId}`,
    JSON.stringify(dataToStore)
```

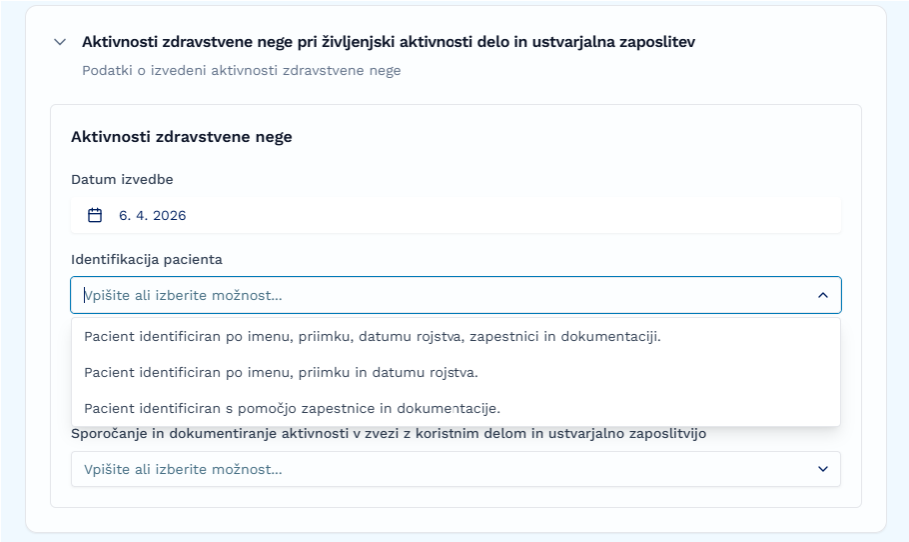


Slika 5: Primer spustnega seznama za izbiro podatkov o pacientu

```
  );
};

// Branje: ob vsaki uporabi preverimo, ce je vnos se veljaven
const loadFormState = (formId) => {
  const stored = localStorage.getItem(`mediform-${formId}`);
  if (!stored) return null;
  const parsed = JSON.parse(stored);
  // Preverimo casovni zig
  if (Date.now() > parsed.expires) {
    localStorage.removeItem(`mediform-${formId}`); // Brisanje
    return null;
  }
  return parsed.state;
};
```

Izsek kode 2: Shranjevanje in branje stanja obrazca z rokom veljavnosti 60 dni

Kot je razvidno iz Izseka kode 2, vsak shranjeni dokument vsebuje poleg vsebine tudi



Slika 6: Primer večkratne izbire aktivnosti zdravstvene nege

časovni žig poteka (`expires`). Ob vsakem naslednjem odprtju aplikacije se preveri, ali je shranjeni podatek še znotraj veljavnega obdobja dveh mesecev. Če čas poteče, se podatek samodejno izbriše. To je bistveno z vidika varstva osebnih podatkov, saj bi se lahko v primeru, da teh podatkov ne pobrišemo, ti lahko ramznoževali naprej preko varnostnih kopiranj telefona.

Uporabnik lahko tudi ročno izbriše shranjene podatke, kar se zgodi vedno, ko se odjavi iz aplikacije z gumbom (`Odjavi se`).

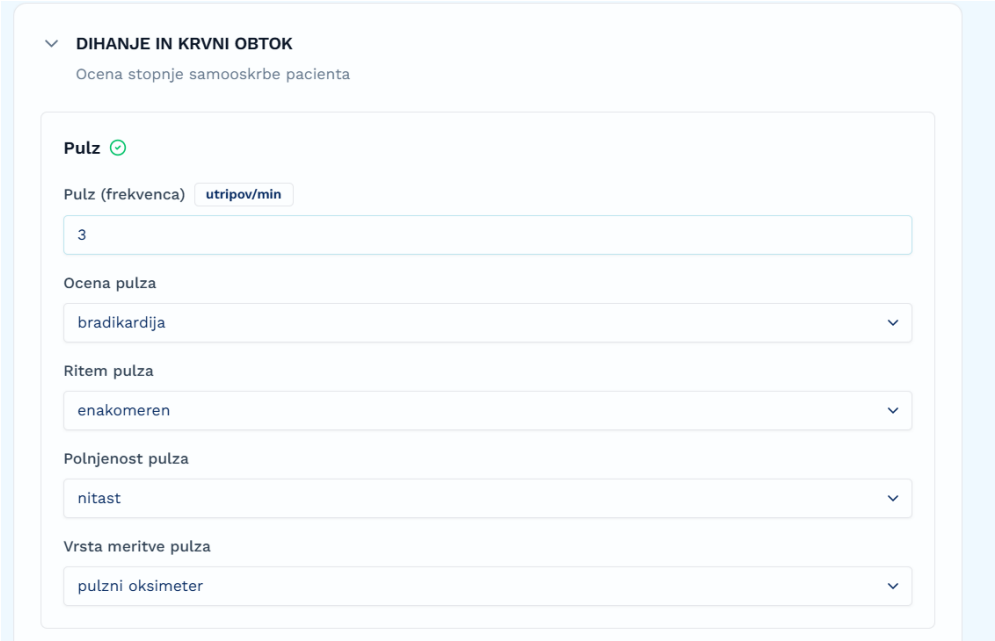
3.4.3 Prikaz napredka obrazca

```
const isSubcategoryComplete = (categoryId, subcategoryId) => {
  if (!list) return false;

  const subcategory = list.categories[categoryId]?.
  subcategories[subcategoryId];
  if (!subcategory) return false;

  const elements = Object.values(subcategory.elements);
  if (elements.length === 0) return false;

  return elements.every((element) => {
    // Tabela je dokoncana, ce vsebuje dejanski vnos
    if (element.type === "table") return isTableModified(element);
```

```
14
15      // Pri bool zadostuje eksplicitna vrednost true ali false
16      if (element.type === "bool") {
17        return element.value === true || element.value === false;
18      }
19
20      // Vecizbirna polja: vsaj ena izbrana moznost
21      if (Array.isArray(element.value)) return element.value.length > 0;
22
23      // Ostala polja: vrednost ne sme biti prazna
24      return (
25        element.value !== null &&
26        element.value !== undefined &&
27        element.value !== ""
28      );
29    });
30  };
31
32  const isCategoryComplete = (categoryId) => {
33    if (!list) return false;
34
35    const category = list.categories[categoryId];
36    if (!category) return false;
37
38    return Object.keys(category.subcategories).every((subcategoryId) =>
39      isSubcategoryComplete(categoryId, subcategoryId)
40    );
41  };
42
43  // Prikaz kljukice v naslovu odseka
44  {isCategoryComplete(categoryId) && (
45    <CheckCircle2 className="h-5 w-5 text-green-500 flex-shrink-0" />
46  )}
```

Izsek kode 4: Preverjanje dokončanosti odsekov in prikaz kljukice

Kot je razvidno iz Izseka kode 4, sistem za vsak pododsek preveri, ali so izpolnjena vsa pripadajoča polja, nato pa enako logiko uporabi še na ravni celotnega odseka. Ko je

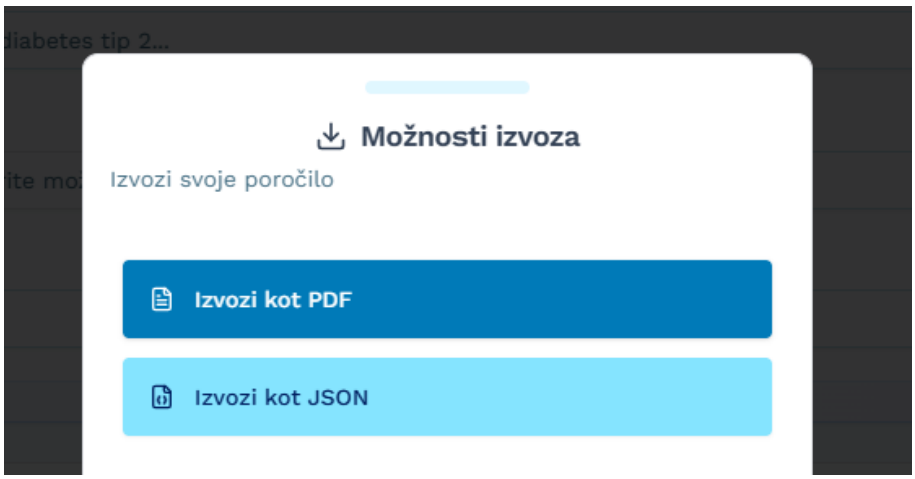


Slika 7: Prikaz dokončanega dela obrazca, kjer sistem z zeleno kljukico označi zaključen odsek

odsek dokončan, se v uporabniškem vmesniku prikaže zelena kljukica. Tako dijak hitro vidi, kateri deli poročila so zaključeni, in zmanjša možnost, da bi ob oddaji spregledal katero izmed obveznih vsebin.

3.4.4 Generiranje PDF in varovanje osebnih podatkov pri izvozu

Generiranje PDF je ena najpomembnejših funkcij aplikacije. Izsek kode 5 prikazuje, kako je implementiran izvoz poročila z varnim poimenovanjem datoteke.

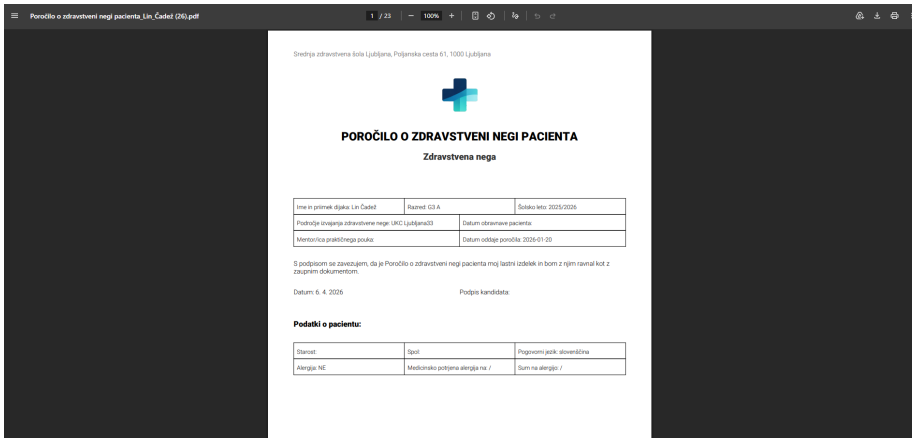


Slika 8: Možnosti izvoza poročila

```
// OPOMBA: To je skrajsan izsek. Celotna implementacija PDF izvoza
// vsebuje se dodatno logiko za naslovnico, prelom strani,
// izpis kategorij, podkategorij, tabel in
// koncno oblikovanje dokumenta.

const exportToPdf = async (formData) => {
  const pdfDoc = await PDFDocument.create();
  pdfDoc.registerFontkit(fontkit);

  // Nalozimo pisavo
  const fontBytes = await fetch("/fonts/Roboto-Regular.ttf")
    .then((res) => res.arrayBuffer());
  const font = await pdfDoc.embedFont(fontBytes);

  const page = pdfDoc.addPage();
  const { height } = page.getSize();

  page.drawText("MediForm - porocilo o zdravstveni negi", {
    x: 50,
    y: height - 50,
    size: 14,
    font,
  });
  page.drawText(`Datum: ${formData.date}`, {
    x: 50,
    y: height - 80,
    size: 11,
    font,
  });
  page.drawText(`Starost pacienta: ${formData.age} let`, {
    x: 50,
    y: height - 100,
    size: 11,
    font,
  });

  // GDPR: Ime datoteke ne vsebuje osebnih podatkov pacienta!
  const now = new Date();
```

```
  const dateStr = now.toISOString().slice(0, 10);
  const timeStr = now.toTimeString().slice(0, 5)
    .replace(":", "-");
  const fileName =
    `OBRAVNAVA-${dateStr}-${timeStr}-${formData.age}let.pdf`;

  const pdfBytes = await pdfDoc.save();
  const blob = new Blob([pdfBytes], { type: "application/pdf" });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = fileName;
  a.click();
};
```

Izsek kode 5: Generiranje PDF in anonimno poimenovanje datoteke (GDPR)



Slika 9: Primer generiranega PDF poročila

Kot je razvidno iz Izseka kode 5, je ime PDF-datoteke sestavljeno iz datuma, ure in starosti pacienta, ne pa iz imena ali priimka. To je pomembno z vidika GDPR. Ker uporabniki pri ustvarjanju dokumenta lahko nehote vpišejo pravo ime pacienta, sem uvedel shemo poimenovanja `OBRAVNAVA-DATUM-URA-STAROST`. S tem se zmanjša možnost razkritja osebnih podatkov.

3.4.5 Odzivni prikaz na telefonu

Izsek kode 6 prikazuje, kako je z razredi Tailwind CSS dosežena odzivna postavitev za telefon in računalnik.

```
<div className="grid grid-cols-1 gap-4 md:grid-cols-2">
  <input
    className="w-full rounded-lg border p-3"
    placeholder="Vnesi vrednost..."
  />
  <button
    className="w-full rounded-lg bg-sky-600 p-3
               text-white hover:bg-sky-700 md:w-auto"
    onClick={handleSave}
  >
    Shrani
  </button>
</div>
```

Izsek kode 6: Odzivna postavitev z razredi Tailwind CSS

Razreda `grid-cols-1` in `md:grid-cols-2` določata prilagoditev prikaza glede na velikost zaslona. Na manjših napravah je vsebina prikazana bolj ozko in pregledno, na večjih zaslonih pa se razporeditev razširi. Gumb je na telefonu čez celotno širino, kar je posebej koristno med delom na praksi.

3.5 Hranjenje podatkov, GDPR in pravne zadeve

3.5.1 Splošno o shranjevanju

Aplikacija MediForm ne pošilja nobenih podatkov na strežnik. Vsi podatki ostanejo izključno v **lokalnem pomnilniku brskalnika** (`localStorage`) na napravi uporabnika. To pomeni, da nobena tretja oseba, vključno z razvijalcem aplikacije, nima dostopa do vnesenih podatkov.

Samodejno shranjevanje ob vnosu: Aplikacija je zasnovana tako, da se vsak

vpis v obrazec **takoj** shrani v `localStorage` — še preden uporabnik klikne gumb za shranjevanje ali zapusti stran. To pomeni, da se podatki ohranijo tudi ob nenamernem zaprtju zavihka ali izgubi internetne povezave.

3.5.2 Skladnost z uredbo GDPR

Uredba GDPR (Splošna uredba o varstvu podatkov) določa, da morajo biti osebni podatki shranjeni le toliko časa, kolikor je nujno potrebno, ter zavarovani pred nepooblaščenim dostopom.

Podatki o pacientih v zdravstveni negi so **občutljivi osebni podatki** v smislu 9. člena uredbe GDPR. Aplikacija MediForm je zasnovana tako, da identifikacija posameznega pacienta ni mogoča: v aplikaciji se ne shranjuje ime ali priimek, temveč le starost, datum in zdravstveni parametri. Izjema je polje za poimenovanje dokumenta, kjer **uporabniki nehote vpišejo pravo ime in priimek pacienta**. Tega ne moremo popolnoma preprečiti z zahtevami v vmesniku, zato je ključno, da se takšno ime nikoli ne prenese v PDF ali deli naprej.

Zasnova varstva podatkov temelji na naslednjih načelih:

- **Minimizacija podatkov**: shranjuje se le tisto, kar je nujno za izpolnitev obrazca.
- **Omejitev hrambe**: vsak dokument ima rok veljavnosti dveh mesecev (60 dni). Po poteku se samodejno izbriše.
- **Lokalna hramba**: podatki niso nikjer drugje kot v lokalnem pomnilniku brskalnika te naprave.
- **Brisanje ob odjavi**: ob odjavi se izbrišejo piškotki seje, kar prepreči dostop nepooblaščenim osebam.
- **Anonimno poimenovanje PDF**: izvozene datoteke so poimenovane po formatu `OBRAVNAVA-DATUM-URA-STAROST`, ne po imenu pacienta.

3.5.3 Delovanje localStorage in upravljanje z veljavnostjo dokumentov

Vsak shranjeni dokument vsebuje polje `expires` z Unix-časovnim žigom (milisekunde od 1. 1. 1970). Ob vsakem zagonu aplikacije se za vsak shranjeni dokument preveri, ali je trenutni čas (`Date.now()`) večji od vrednosti `expires`. Če je, se dokument samodejno izbriše iz `localStorage`.

3.6 Testiranje aplikacije

Testiranje aplikacije sem izvedel v dveh fazah. V prvi fazi sem funkcionalnost preveril sam z ročnim testiranjem — preizkusil sem izpolnjevanje obrazca, shranjevanje stanja v `localStorage`, samodejno brisanje potečenih dokumentov ter izvoz v PDF. Posebno pozornost sem namenil delovanju na mobilnih napravah, saj je bila zahteva naročnice, da aplikacija deluje na telefonu.

V drugi fazi so aplikacijo preizkusile dijakinje Srednje zdravstvene šole Ljubljana med prakso v bolnišnici. Na podlagi njihovih povratnih informacij sem odkril in odpravil več napak: popravil sem način shranjevanja stanja ob prijavi, prilagodil nekatere elemente obrazca za lažjo uporabo na manjšem zaslonu ter dopolnil vsebino kontrolnega seznama glede na dejanske potrebe z dela na terenu. Testiranje v realnem okolju se je izkazalo za najpomembnejši del zagotavljanja kakovosti, saj so uporabniki odkrili težave, ki jih s samostojnim testiranjem nisem mogel predvideti.

3.7 Objava spletne strani

Spletna aplikacija je bila objavljena preko platforme podjetja Vercel. Ker MediForm ne uporablja ločenega strežniškega dela, je bil postopek enostaven: priprava produkcijskega `build` paketa in objava na domeni `https://mediform.cadez.eu/`. Po objavi sem preveril delovanje obrazca, shranjevanja in izvoza PDF.

4 Zaključek

Razvoj aplikacije MediForm mi je pokazal, da je mogoče z ustreznim načrtovanjem in dobrim sodelovanjem z naročnikom razviti uporabno digitalno rešitev za šolsko okolje. Aplikacija je nastala kot odgovor na konkreten problem: dijaki so poročila izpolnjevali tik pred rokom in si podatke izmislili.

Ugotovil sem, da je bila najpomembnejša dobra komunikacija s stranko. Sproti sem preverjal, kaj naročnica dejansko potrebuje, in na podlagi njenih pripomb popravljal aplikacijo. Pri tehnični izvedbi sem se srečal z več izzivi, predvsem pri organizaciji obrazcev, samodejnem shranjevanju podatkov, varovanju zasebnosti in pripravi izvoza v PDF. Vsak problem sem reševal postopno skupaj z naročnico.

Na koncu sem ugotovil, da MediForm uporabnikom olajša delo in — kar je najpomembnejše — omogoča bolj sprotno in resnično izpolnjevanje dokumentacije.

Viri

Firebase. (2026). *Firebase Documentation*. Dostopno prek: https://firebase.google.com/docs/

pdf-lib. (2026). *pdf-lib Documentation*. Dostopno prek: https://pdf-lib.js.org/

React. (2026). *React Documentation*. Dostopno prek: https://react.dev/

Vercel. (2026). *Vercel Documentation*. Dostopno prek: https://vercel.com/docs

Priloge

Spletno mesto in GitHub

Spletna aplikacija MediForm je dostopna na naslovu: `https://mediform.cadez.eu/`

Izvorna koda projekta je dostopna na GitHubu: `https://github.com/lin-cadez/MediForm`

Izjava o avtorstvu

Izjavljam, da je seminarska naloga *Izdelava spletne strani za naročnika* v celoti moje avtorsko delo, ki sem ga izdelal samostojno s pomočjo navedene literature in pod vodstvom mentorja.

Ljubljana, april 2026 Lin Čadež

(podpis)